Notes de frais sous chaos

B5 — Workflow agentique résilient et idempotent

Groupe transformers · @nathan138 · @Marko075 · @TODO-pseudo-alexandre

MSMIN5IN52 — IA générative et chatbots · EPF · 28/10/2026

Le problème

Une note de frais en texte libre traverse 5 étapes :

1. **Réception — validation, dédoublonnage**
2. **Extraction — LLM → JSON validé par schéma**
3. **Contrôle — plafonds, dates, doublon métier**
4. **Virement — effet externe (argent)**
5. **Notification — effet externe (e-mail)**

Fenêtre critique : le service a payé… et le worker meurt avant d'écrire « virement terminé ».

Au redémarrage, on repaie ?

Pourquoi « réessayer » ne suffit pas

- Une file de messages livre au moins une fois : un même message peut arriver deux fois.
- Un timeout ne dit pas si l'effet a eu lieu : le service a peut-être déjà payé.
- Un retry sans précaution = double virement.

Le verrou n'est pas le nombre d'agents : c'est la sémantique d'exécution (un effet, une seule fois).

Notre solution : trois mécanismes

Mécanisme	Ce qu'il garantit
Étapes persistées (SQLite WAL, transactions)	un worker relancé reprend à la 1re étape non finie
Clés d'idempotence request_id:étape	le service rejoue sa réponse au lieu de refaire l'effet
Cache LLM (modèle + prompt + texte)	une extraction déjà payée n'est jamais redemandée

- **Compensation : notification impossible → remboursement → COMPENSATED**
- **Réconciliation : retries épuisés → on demande au service, par la clé, si le paiement a eu lieu**

Architecture

- **Superviseur : lance le worker dans un processus séparé, le relance après chaque crash**
- **Chaos : tue le worker (psutil) au hasard ; points de crash nommés pour les tests**
- **Deux bases : workflow.db (état) ≠ services.db (effets externes, registre, clés)**
- **LLM : Semantic Kernel → OpenRouter (qwen/qwen3.8-flash), ou simulé / rejoué hors ligne**
- **Baseline naïve : même pipeline, mêmes retries, sans état ni clé**

Protocole de la campagne

- 40 notes synthétiques : 28 nominales, 9 refus attendus, 3 compensations attendues
- 15 % des messages livrés deux fois
- 5 seeds × 2 modes, worker tué toutes les 0,3–1,2 s
- Pannes de services : 500 (10 %), timeout après l'effet (5 %)
- LLM simulé : JSON invalide (5 %), indisponible (3 %)

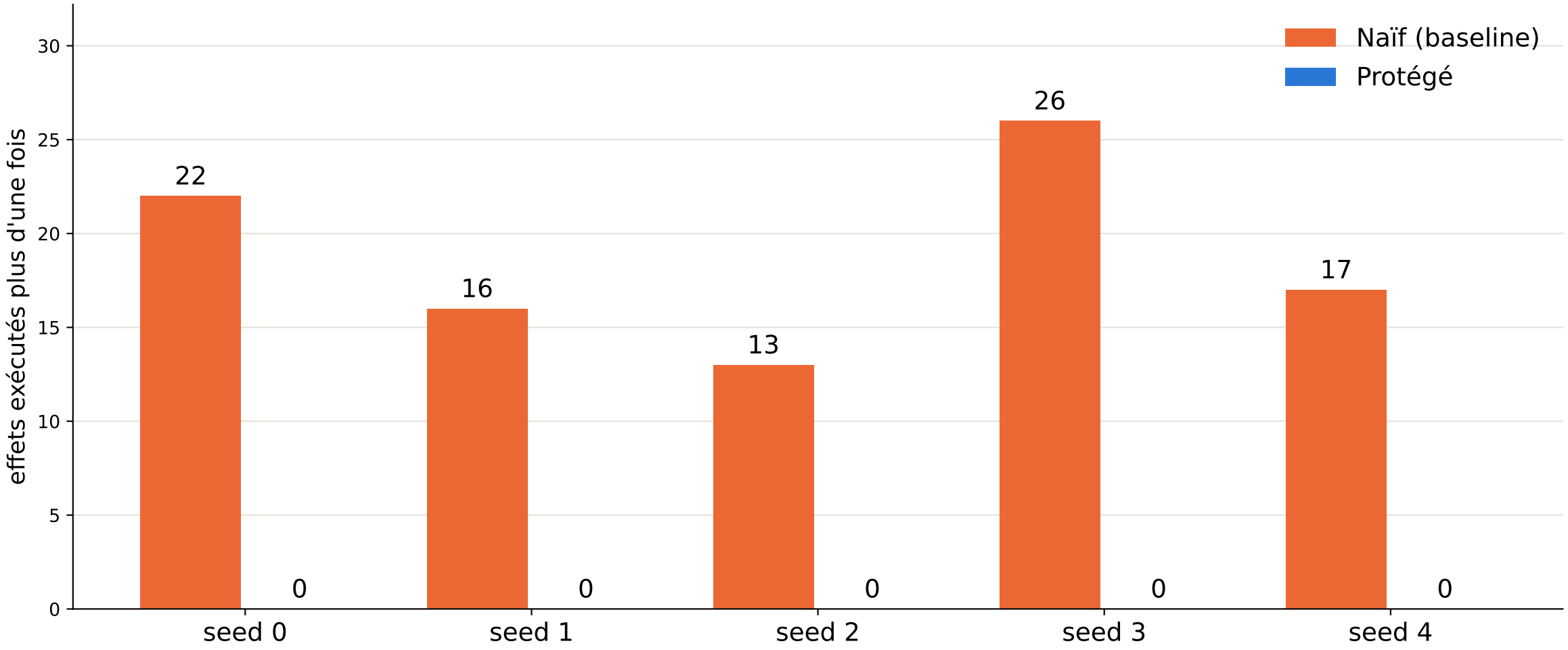
≥ 40 crashs par mode — 201 pannes (naïf) / 146 (protégé)

Résultats

5 runs par mode	Naïf	Protégé
Crashs injectés	65	49
Complétion	100 %	100 %
Effets en double	**94**	**0**
Demandes incohérentes	44 / 200	0 / 200
Étapes refaites	278	0
Appels LLM	303	213
Reprise p50 / p95	0,36 / 0,40 s	0,41 / 0,47 s

La complétion ne suffit pas : le naïf « finit », mais dans un état faux.

Effets en double, par seed



Vrai LLM : la preuve

- 10 demandes variées (FR/EN, EUR/USD/GBP/CHF, refus, compensation)
- **10 appels réels, 100 % des champs corrects (montant, devise, catégorie, date)**
- Crash forcé juste après l'appel LLM → reprise via le cache, sans nouvel appel
- 0 doublon · budget : 11 appels / 150

Limites et suite

- Services simulés : en vrai, il faut un service qui accepte les clés (Stripe) — sinon registre côté client
- Un seul worker : plusieurs workers → verrou par demande (bail)
- Temps de reprise = surtout le démarrage de Python
- Qualité d'extraction mesurée sur 10 appels seulement

Suite : workers concurrents, saga multi-étapes, OpenTelemetry

Démo

```
python -m demo                          # naïf (rouge) puis protégé (vert), ~30 s
python -m notesfrais dashboard          # « Tuer le worker maintenant »
python -m notesfrais replay             # plan B sans rien exécuter
```

Rapport : results/report.html · Analyse : notebooks/analyse.ipynb

Code produit avec l'assistance de Claude Code, piloté et relu par le groupe.